

Contents

Enterprise Security (Cloud and IoT).....	1
Secure Private Cloud Hosting Environment Report	1
1. Problem Statement	1
2. Architecture Overview	2
3. Security Controls Justification.....	5
4. Security Testing and Validation.....	6
5. Residual Risks and Limitations.....	6
6. Conclusion.....	7
7. References (Harvard style).....	7

Enterprise Security (Cloud and IoT) Secure Private Cloud Hosting Environment Report

1. Problem Statement

Organisations these days that handle regulated or sensitive data face increasing security challenges because of the shift towards distributed systems and containerised environments. The usual perimeter-based security models that assume that internal networks are naturally trusted are no longer good enough. Attackers can exploit vulnerabilities like exposed services, weak authentication mechanisms and make sure communication channels to gain access to internal systems.

This ca aims to address these challenges by designing and implementing a secure private cloud hosting environment using Docker on an Ubuntu Server VirtualBox machine. The environment provides a web-facing service supported by internal components while making sure that access is restricted, communications are encrypted and system activity is continuously monitored.

The design is based on two fundamental security principles:

- Zero Trust Architecture: No user, device or service is trusted by default. All access requests must be verified through authentication and authorisation mechanisms.
- Defence in Depth: Multiple layers of security controls are implemented across the system so that if one layer fails then others continue to provide protection.

The key challenges addressed in this ca are:

- Preventing direct access to backend services like databases
- Making sure secure communication using HTTPS and TLS encryption
- Implementing centralised identity and access management
- Detecting vulnerabilities in container images and web services
- Providing visibility through logging, monitoring and metrics

By addressing these challenges, the ca demonstrates how a secure and controlled hosting environment can be implemented using container-based technologies.

2. Architecture Overview

The system is deployed on a VirtualBox Ubuntu Server and uses Docker Compose to orchestrate multiple containers. The architecture is designed to enforce clear separation of responsibilities and minimise unnecessary exposure of services.

At the core of the system is Traefik, which acts as a reverse proxy and central ingress point. All incoming traffic is routed through Traefik making sure that backend services are not directly exposed to external users.

The main components of the architecture include:

- Traefik – Handles routing of incoming requests, TLS termination and enforces HTTPS
- Keycloak – Provides authentication and authorisation through identity-based access control
- Application (Nginx) – Represents the web-facing service accessed by users
- PostgreSQL – Stores application data and is isolated from external access
- Dozzle – Provides real-time logging of container activity
- Prometheus – Collects system and service metrics
- Grafana – Visualises metrics collected by Prometheus
- CrowdSec – Detects malicious activity based on log analysis
- Cloudflare Tunnel – Enables secure external access without exposing ports

To enforce network isolation, two Docker networks are implemented:

- frontend_net – Contains services exposed to users via Traefik
- backend_net – Contains internal services like the database

Traefik is connected only to the frontend network while backend services are isolated. This makes sure that if the web-facing service is compromised, attackers cant directly access internal components.

This layered architecture establishes clear trust boundaries between external users, the reverse proxy and internal services.

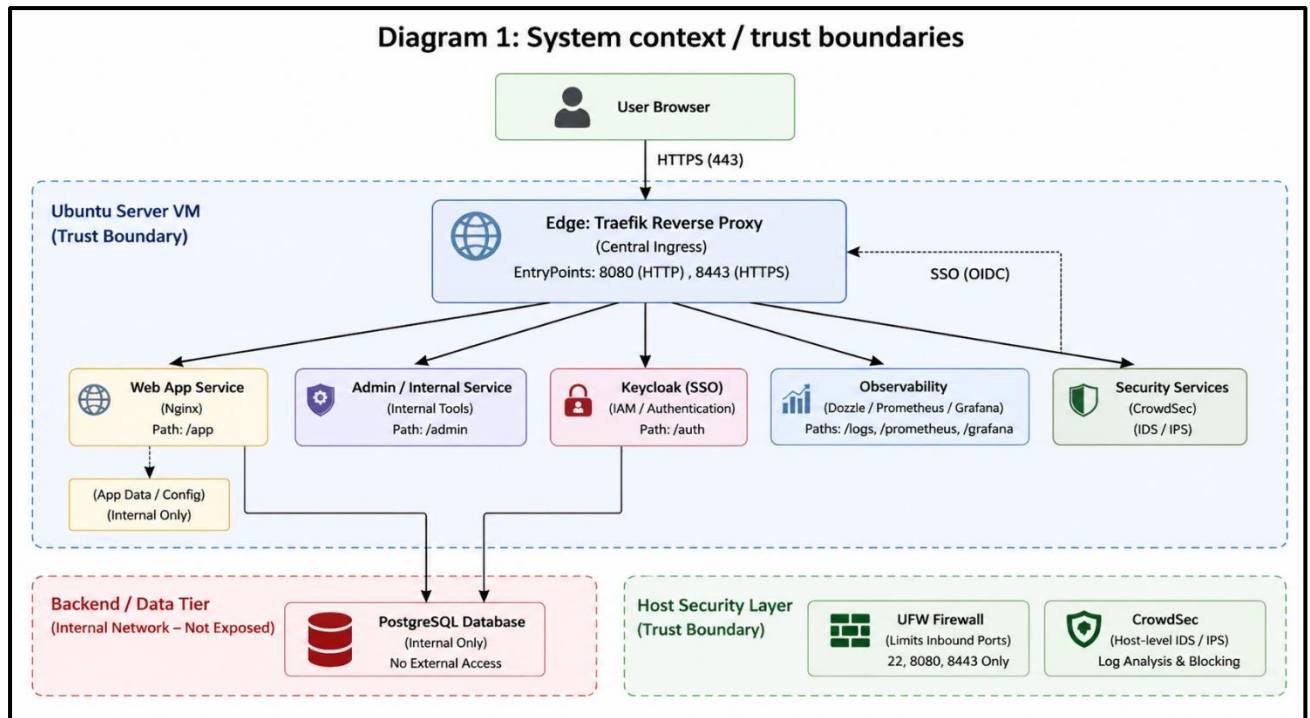


Figure 1: System context and trust boundaries of the secure hosting environment.

Figure 1 shows the overall system context, showing the separation between external users, the reverse proxy and internal services. It also shows how trust boundaries are enforced and how identity management (Keycloak) sits between users and protected services.

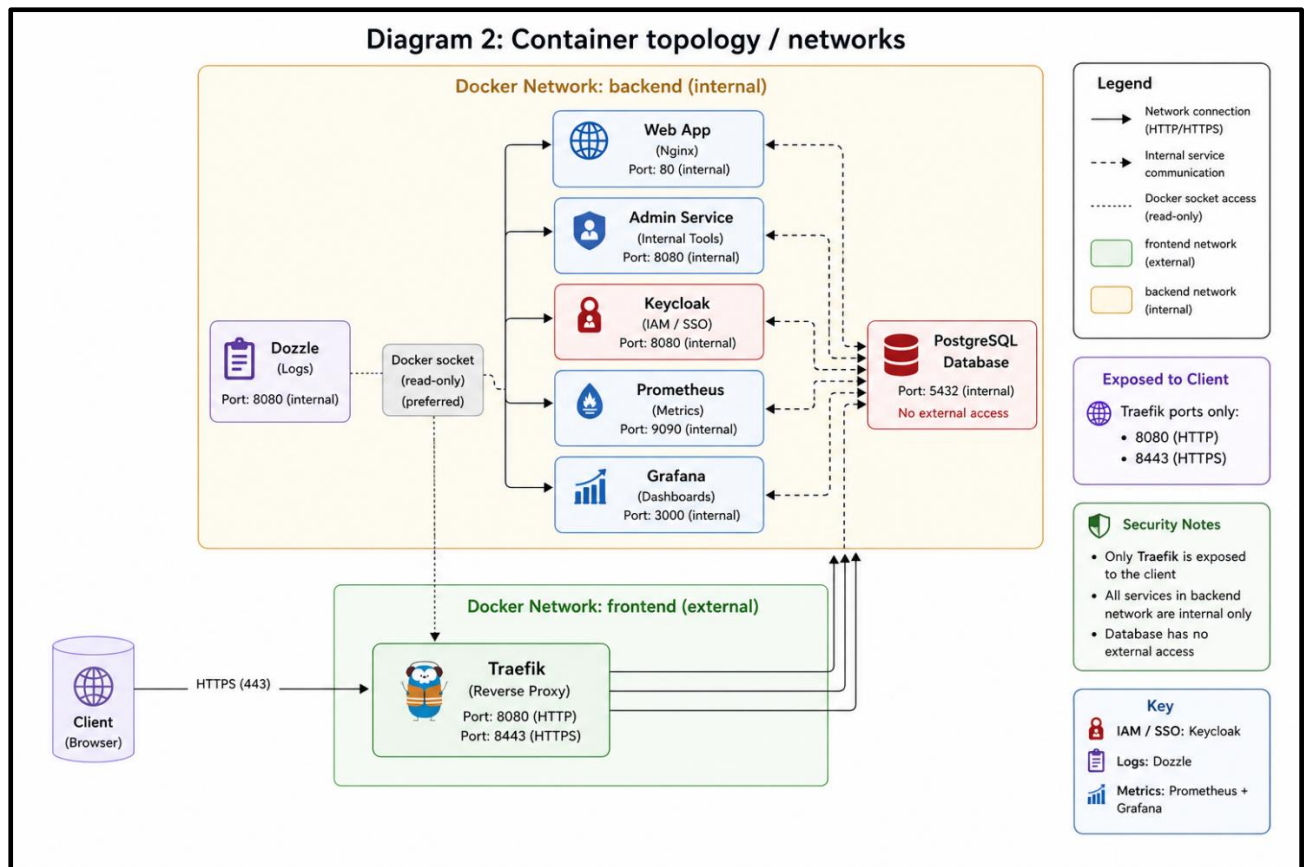


Figure 2: Container topology and network segmentation within the Docker environment.

Figure 2 shows the container-level architecture including the frontend and backend networks. It shows that only Traefik is exposed to external traffic, while internal services like the database are isolated. It also shows where logging, monitoring and IAM components operate in the system.

The architecture also enforces controlled service communication and dependency boundaries. Each container only communicates with the services it requires. For example the application container communicates with the database over the backend network while external requests are strictly routed through Traefik. This prevents direct service-to-service exposure and makes sure that all access paths are intentional and auditable.

Using containerisation strengthens isolation because each service runs in its own environment with limited privileges. This reduces the risk of one compromised service affecting others. Environment variables and service configurations are managed through Docker Compose allowing for consistent and reproducible deployments.

From a scalability perspective the architecture supports adding new services without significantly increasing risk. Any new service can be placed into the appropriate network and exposed only through Traefik if required. This keeps a consistent security model while allowing the system to grow.

Overall the architecture balances security, modularity and scalability making sure that the environment stays controlled while still being flexible enough for future growth.

3. Security Controls Justification

Reverse Proxy and Central Ingress

Traefik is configured as the sole entry point into the system. By centralising ingress all incoming traffic can be controlled, monitored and filtered. This reduces the attack surface and makes sure that no service is exposed unnecessarily. This approach aligns with the Principle of Least Exposure which aims to minimise the number of accessible entry points.

HTTPS and TLS Enforcement

All external communication is secured using HTTPS. TLS encryption makes sure of confidentiality and integrity of data in transit protecting against interception and tampering. Enforcing HTTPS also prevents users from accessing services over insecure HTTP connections.

Network Segmentation and Isolation

The use of separate Docker networks enforces strict segmentation between services. Backend services like PostgreSQL are not accessible from the frontend network, stopping direct access from external users.

This approach supports Zero Trust principles by making sure that access is not granted based on network location. Instead communication between services is clearly defined and restricted.

Identity and Access Management (IAM)

Keycloak is used for authentication and authorisation. Users must authenticate before accessing protected services and access is controlled using roles.

This makes sure that access decisions are based on identity. It enables the enforcement of policies like role-based access control (RBAC) which is important in regulated environments.

Logging (Dozzle)

Dozzle provides real-time logging of container activity. This allows administrators to monitor system behaviour and quickly find issues or suspicious activity.

Logging is a critical component of Defence in Depth as it provides visibility into system operations and supports incident investigation.

Monitoring (Prometheus and Grafana)

Prometheus collects metrics from services and Grafana visualises these metrics. This enables monitoring of system health and performance.

Metrics supports proactive detection of issues like service failures or unusual activity patterns. For example, the up query in Grafana confirms whether monitored services are operational.

Host-Level Security Controls

The Ubuntu firewall (UFW) is configured to allow only necessary ports like SSH and HTTPS. This adds an extra layer of protection at the host level making sure that even if container configurations are incorrect the host stays protected.

Intrusion Detection (CrowdSec)

CrowdSec analyses logs to detect patterns of malicious behaviour like repeated failed login attempts. This provides a dynamic security layer that can find threats based on behaviour rather than static rules.

Secure External Access (Cloudflare Tunnel)

Cloudflare Tunnel is used to expose the application externally without opening inbound ports. This reduces the risk of direct attacks on the host and stops the need for port forwarding.

4. Security Testing and Validation

Image Scanning (Trivy)

Trivy was used to scan container images for known vulnerabilities. The scan found low and medium severity vulnerabilities which are normal for publicly available base images.

No critical vulnerabilities were found. Mitigation strategies include using minimal base images like Alpine Linux and regularly updating containers to make sure that security patches are applied.

OWASP ZAP Baseline Scan

A baseline scan was conducted using OWASP ZAP against the application endpoint. The scan found warnings including missing HTTP security headers like Content Security Policy (CSP), Strict Transport Security (HSTS) and X-Content-Type-Options.

5. Residual Risks and Limitations

Despite the implementation of multiple security controls some risks remain because of the limitations of the lab environment.

- The database is not replicated creating a potential single point of failure
- Self-signed TLS certificates are used which are not trusted by browsers
- Role-based access control is basic and does not reflect complex enterprise policies
- CrowdSec is not fully integrated with automated blocking mechanisms
- Monitoring does not include advanced alerting

In a real-world deployment these limitations would be addressed by implementing high availability using trusted certificate authorities and enforcing stricter access control policies.

6. Conclusion

This ca demonstrates the implementation of a secure private cloud hosting environment using container technologies. By applying Zero Trust and Defence in Depth principles the solution makes sure that security is enforced across multiple layers, including network segmentation, identity management, encryption, monitoring and intrusion detection. While some limitations exist because to the lab environment, the architecture provides a strong foundation for secure and scalable deployments.

7. References

OWASP (2024) *OWASP ZAP Documentation*. Available at: <https://www.zaproxy.org>

Docker (2024) *Docker Security Overview*. Available at: <https://docs.docker.com/security/>

Cloudflare (2024) *Cloudflare Tunnel Documentation*. Available at: <https://developers.cloudflare.com>

Prometheus (2024) *Monitoring Systems*. Available at: <https://prometheus.io>